

FocusFlow — Technical Design

Version: 1.0

Date: 2 October 2026

Status: Draft for review

Scope: the 10-day prototype (Section 1A of the implementation plan)

Reader: someone who does not write code. Every technical choice below is explained in plain words, and every choice exists to serve one of three rules.

How to read this document

Section 1 explains the tools. Sections 2 to 5 explain how the data is shaped and why that shape is hard to regret. Section 6 is your build schedule. Section 7 is the two decisions you need to make.

One thing to know before you start: **this prototype is a foundation, not a sketch**. Every dropped feature — goals, capture, focus mode, routines, onboarding, calendar, insights, offline — has a named place to land later. Section 5 lists them all. The journal is the one exception: a **minimum** version ships on Day 9, and only its prompts, search and history views are deferred.

1. The recommended tech stack

The guiding question was: *what is the smallest number of things I have to learn and keep running, that can still grow into the real app?*

The answer is a **web app that runs in a browser**.

Layer	What to use	Why, in plain words
Language	TypeScript	JavaScript with a safety net. It spots your AI assistant's typos and mistakes <i>before</i> you run the app, instead of after. If two pieces of code disagree, it tells you exactly where.
Framework	Next.js	Turns a folder of text files into a working website. You type <code>npm run dev</code> , it starts, and you open a browser. It is the most-used web framework in the world, which means AI assistants know it extremely well — fewer strange errors.
Database	PostgreSQL (hosted on Neon, free tier)	A spreadsheet that behaves like a proper database: it will not let you save a task pointing at a goal that does not exist. Free tier is generous and does not need a credit card.
The layer between code and database	Prisma	You describe your tables in one readable file, in plain English-like lines. Prisma turns that description into a real database and gives you simple functions to read and write. You never write raw database commands by hand.
Styling	Tailwind CSS	Short class names instead of long stylesheet rules. It matters here because the day view is a dense timeline, and this is the fastest reliable way to lay that out.
Where it runs	Your own computer (Node.js)	The app runs as an ordinary program on your machine. No account, no card, no terms of service, and no cost that can ever change. Testers reach it over your Wi-Fi, or through a free Cloudflare Tunnel link.
Time handling	date-fns + date-fns-tz	Handles time zones and daylight saving for you, so 9am stays 9am even when the clocks change.

1.1 The most important trade-off, stated honestly

The earlier implementation plan recommended React Native with offline-first storage. **I am recommending a browser app instead, and this deviates from that plan on purpose.**

Here is the reasoning, and you should overrule me if you disagree.

What you gain. A browser app has no app store, no install, no build step on your machine, no signing certificates. You change one line of code and refresh the page. That is a very large advantage when you are learning, and when your test needs three real people using the thing by Friday.

What you give up. A browser app cannot block other apps, so true focus mode needs a native app later. And offline use needs real work later, because browsers make offline hard.

Why the second one is survivable. Here is the part that matters: the expensive parts of this app are **not** the parts a browser is bad at. The scheduler, the time-zone handling, the constraint rules, and the data model are all ordinary TypeScript. They are not tied to a browser. When you move to a native app, that code moves across unchanged — it is the same language, and it has no browser-specific pieces.

What you would be rewriting later is the storage layer and the screens. That is exactly why Section 2 puts every read and write behind one folder. So the honest summary is:

Choosing a browser app now costs you a later UI and storage effort. It does not cost you the scheduling engine, which is the part that would be genuinely painful to redo.

If you would rather build native now and accept that the first ten days are mostly setup friction, say so. The data model in Section 2 is identical either way, so nothing in this document is wasted.

1.2 Why these choices specifically

- **TypeScript over JavaScript** — because you are not writing the code, you are reviewing it. TypeScript turns silent mistakes into loud errors, which is exactly what you want when someone else is typing.
- **Next.js over anything else** — you will describe a screen in a sentence and get a working screen back. Its popularity is a feature here.
- **PostgreSQL over a simpler database** — SQLite lives inside a file on your computer, which breaks the moment you deploy to the internet. Starting hosted means you never migrate, and you can share a link with testers on day one.
- **Prisma over writing database code** — you edit one readable description file, run one command, and the database updates itself. No hand-written SQL to break.
- **One repository, one app** — a second codebase to keep in sync is a second thing to forget about.

2. Database design

This is the heart of the document, and the part that most needs to be right on day one.

2.1 How to read the schema

Each table below is described in three parts: what a row stores, which other tables it points at, and **which rule it enforces**. Read the rule column especially — it is the whole point.

2.2 The ten tables

User — who is using the app

- One row per person. Stores a display name, a time zone, and a creation date.
- **Why the time zone lives here:** it is a property of the person, not of any one moment. Section 2.4 explains how this is used.

Goal — a meaningful objective

- Title, the reason behind it, an optional target date, a status.
- **Empty on day one, deliberately.** The table exists from the start with the columns it will always need, so adding goals later means building screens, not moving data.

Project — a piece of a goal

- Title, status, and a required link to a `Goal`.
- **Also empty on day one.** A project always belongs to a goal, which is enforced by the database itself.

Task — a thing to do

The most important table. Stores:

- **What it is:** title, optional notes
- **How long and how hard:** an estimate in minutes, and a difficulty of easy, medium, or hard
- **Where it belongs:** `goalId` and `projectId`, **both optional and both empty right now — Rule A**
- **What state it is in:** `inbox`, `ready`, `scheduled`, `done`, or `dropped` — the future inbox lives here, because the field exists now
- **How often it repeats:** `recurrenceRule`, optional, in the standard `RRULE` text format, so routines need no new column later
- **Timing:** an optional deadline, and `actualMinutes` left empty for now, filled in later when someone finishes the task
- **Protection:** `isPinned`, so a task you deliberately placed somewhere cannot be moved by a later re-plan

Connections: belongs to a `User`, and *optionally* points at a `Goal` and a `Project`.

Rule enforced: goals and projects can be switched on without touching this table.

Block — time that is already spoken for

This is where **Rule B** lives, and it is worth reading carefully.

A block is a fixed piece of time: a meeting, a doctor's appointment, lunch. It stores:

- A start and an end
- What it is for (a title)
- **Where it came from:** `source` is `manual`, `calendar`, or `system`
- What kind it is: a meeting, personal time, or a break
- Whether it can move
- An optional `externalId` and `calendarProvider`, for the day a real calendar is connected
- An optional `notes` field

Connections: belongs to a `User`.

Rule enforced, structurally: a block has **no column that can point to a task**. There is nowhere to put such a link. A fixed meeting cannot become a task that refuses to move, because the database has no field for it. This is the strongest form of Rule B — it does not rely on anyone remembering a rule.

This table is also why a calendar meeting and a hand-typed meeting are stored **identically**. The `source` column records the history; the scheduler never reads it. One hand-typed meeting and one synced meeting are the same kind of row, with one word differing.

WorkingHour — when the person is available

- Which day of the week, and a start and end as minutes from midnight.
- Stored as a **weekly pattern in local time**, not as a timestamp. "I work 9 to 5, Monday to Friday" is not a moment in time, it is a rule that repeats.

EnergyEntry — how alert the person is

- The hour of the day, a level of high, medium, or low, and where the level came from: `declared` (the person drew it) or `observed` (the app learned it from real behaviour).
- **On day one every row is `declared`**. The `source` column already exists so that observed data can be added later without restructuring — this is the schema shape that makes the Phase 0 decision in Section 7 reversible.

DayPlan — one day's proposal

- The date, whose plan it is, whether it is a `draft`, `accepted`, or `discarded`, and how many times it has been regenerated.
- **Why this is a separate table:** re-planning creates a new proposal, it does not destroy the old one. Keeping every version is what lets the app say "here is what changed and why" instead of silently rearranging someone's day.

PlanItem — a task placed into time

- The day plan it belongs to, the task, a start and an end, and whether it is pinned.
- **Connections:** requires both a `DayPlan` and a `Task`. Neither is optional.
- **Rule enforced:** because a plan item must reference a real task, and a block cannot reference a task at all, the two concepts can never blur together.

PlacementReason — why the scheduler did what it did

This is **Rule C**, stored as facts rather than sentences. One row per task considered, per day plan, holding:

- The outcome: `placed` or `rejected`
- If placed: the start and end
- **The energy level at that hour** — `high`, `medium`, or `low`
- **What the task competed against** — a list of other task ids it was ranked against
- **Which constraint bound it** — one of `energy_match`, `deadline`, `working_hours`, `block_collision`, `priority`, `manual_override`, or `unscheduled`
- If rejected: a short reason code

Why fields and not a sentence. Because when the product grows, the same facts answer questions nobody has asked yet. "Why is this here?" reads the energy level. "Show me every deep-work task scheduled during a low-energy hour" reads two fields and a filter. "What did I refuse to schedule this week?" reads the rejected rows. None of those need new data if the facts are stored. If you store a finished sentence, all three become impossible without re-reading and re-interpreting English.

The screen builds the sentence. So "Placed at 09:00 because your energy is high and it's the hardest task" is assembled from those fields at the moment it is displayed. Change the wording of the app and the sentence changes. Change the logic and every explanation still stays accurate.

JournalEntry — what the person thought about the day

- The date it belongs to, an optional link to that day's `DayPlan`, an optional mood, and the free-form text.
- **Scope:** this is the *minimum* journal, added after review. The PRD's guided prompts (J3), coach follow-ups (J7), search (J5), week and month views (J6) and export (J8) are all deferred. It exists to answer one question during testing: does anyone actually write in this?
- **No AI and no prompts**, deliberately. Both J3 and J7 depend on "the coach", which is not in the prototype, so they cannot be built cheaply here regardless of preference.
- **Why it is a new table rather than a field on `DayPlan`** : a person may write on a day with no plan, and may write more than once. One-to-many, so a separate table.
- **Rule enforced:** an entry can only ever point at a `DayPlan` through `dayPlanId`, exactly as a `PlanItem` does. It never copies task text into itself, so a task renamed later cannot leave a stale copy inside a journal entry.

2.3 The schema, written out

Your AI assistant will type something close to this. Included here so you can see what it is doing.

```

// enums
enum TaskState { INBOX READY SCHEDULED DONE DROPPED }
enum Difficulty { EASY MEDIUM HARD }
enum BlockSource { MANUAL CALENDAR SYSTEM }
enum BlockKind { MEETING PERSONAL BREAK }
enum EnergyLevel { HIGH MEDIUM LOW }
enum EnergySource { DECLARED OBSERVED }
enum BindingConstraint { ENERGY_MATCH DEADLINE WORKING_HOURS
                        BLOCK_COLLISION PRIORITY MANUAL_OVERRIDE UNSCHEDULED }

enum Outcome { PLACED REJECTED }
enum PlanState { DRAFT ACCEPTED DISCARDED }
enum Mood { GREAT GOOD OK FLAT ROUGH } // the minimum journal's only new enum. Five points, not four, so there is a true middle.

model User {
  id String @id @default(cuid())
  name String
  timezone String @default("Europe/London") // IANA name, e.g. "Europe/London"
  goals Goal[]
  projects Project[]
  tasks Task[]
  blocks Block[]
  plans DayPlan[]
  workingHours WorkingHour[]
  energyLevels EnergyEntry[]
  journalEntries JournalEntry[] // added with the minimum journal, Day 9
  createdAt DateTime @default(now())
}

model Goal { // empty on day one, columns final
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id])
  title String
  why String?
  targetDate DateTime?
  state String @default("ACTIVE")
  projects Project[]
  tasks Task[]
}

model Project { // empty on day one, columns final
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id])
  goalId String // required: a project always has a goal
  goal Goal @relation(fields: [goalId], references: [id])
  title String
  state String @default("ACTIVE")
  tasks Task[]
}

model Task {
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id])
  title String
  notes String?
  goalId String? // RULE A: optional, empty now
  goal Goal? @relation(fields: [goalId], references: [id])
  projectId String? // RULE A: optional, empty now
  project Project? @relation(fields: [projectId], references: [id])
}

```

```

estimatedMinutes Int
difficulty      Difficulty @default(MEDIUM)
priority        Int        @default(3)           // 1 highest, 4 lowest
state           TaskState  @default(READY)       // inbox support, Rule-state
recurrenceRule  String?    // routines, e.g. "FREQ=WEEKLY;BYDAY=MO"
deadline        DateTime?
actualMinutes   Int?       // filled in later, for calibration
isPinned        Boolean    @default(false)       // manual moves survive re-planning
planItems       PlanItem[]
reasons         PlacementReason[]
createdAt       DateTime   @default(now())
updatedAt       DateTime   @updatedAt
completedAt     DateTime?
}

model Block {
    // RULE B: no taskId column exists, ever
    id          String      @id @default(cuid())
    userId       String
    user         User       @relation(fields: [userId], references: [id])
    title        String
    kind         BlockKind  @default(MEETING)
    source        BlockSource @default(MANUAL)
    startsAt     DateTime   // always UTC, always a real instant
    endsAt       DateTime
    isMovable     Boolean    @default(false)
    externalId    String?    // calendar sync, day 1 column
    calendarProvider String?
    notes         String?
    createdAt     DateTime   @default(now())
    updatedAt     DateTime   @updatedAt
    @@index([userId, startsAt])
}

model WorkingHour {
    // a weekly pattern, in the user's own timezone
    id          String @id @default(cuid())
    userId       String
    user         User   @relation(fields: [userId], references: [id])
    dayOfWeek    Int    // 0 = Sunday .. 6 = Saturday
    startsMinute Int    // minutes from local midnight
    endsMinute   Int
    @@unique([userId, dayOfWeek, startsMinute])
}

model EnergyEntry {
    id          String      @id @default(cuid())
    userId       String
    user         User       @relation(fields: [userId], references: [id])
    hour         Int        // 0-23, in the user's own timezone
    level        EnergyLevel
    source       EnergySource @default(DECLARED) // OBSERVED arrives later, no restructure
    @@unique([userId, hour, source])
}

model DayPlan {
    id          String      @id @default(cuid())
    userId       String
    user         User       @relation(fields: [userId], references: [id])
    planDate     DateTime   // the calendar date, local to the user
    state        PlanState  @default(DRAFT)
    regenerateCount Int     @default(0)
    items        PlanItem[]
    reasons      PlacementReason[]
}

```

```

journalEntries JournalEntry[] // added with the minimum journal, Day 9
createdAt DateTime @default(now())
@@unique([userId, planDate])
}

model PlanItem {
  id String @id @default(cuid())
  dayPlanId String
  dayPlan DayPlan @relation(fields: [dayPlanId], references: [id])
  taskId String // required: only tasks get placed
  task Task @relation(fields: [taskId], references: [id])
  startsAt DateTime
  endsAt DateTime
  isPinned Boolean @default(false)
}

model PlacementReason { // RULE C: facts only, never a finished sentence
  id String @id @default(cuid())
  dayPlanId String
  dayPlan DayPlan @relation(fields: [dayPlanId], references: [id])
  taskId String
  task Task @relation(fields: [taskId], references: [id])
  outcome Outcome
  startsAt DateTime?
  endsAt DateTime?
  energyAtStart EnergyLevel? // "the energy level at that hour"
  competingTaskIds Json // "what the task competed against"
  bindingConstraint BindingConstraint // "which constraint bound it"
  rejectionReason String?
  createdAt DateTime @default(now())
  @@index([dayPlanId, outcome])
}

model JournalEntry { // added after review: the minimum journal, Day 9
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id])
  date DateTime @db.Date // J1: every entry belongs to a date
  dayPlanId String? // J2: the day's real work, by reference
  dayPlan DayPlan? @relation(fields: [dayPlanId], references: [id])
  mood Mood?
  body String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([userId, date])
}

```

Why `JournalEntry` is as small as it is. It has five fields and no AI, no prompts, and no attachments, because those are what make the full journal a six-to-eight day project. The two requirements it *does* satisfy are the two that matter for testing whether reflection lands at all: **J1**, every entry is attached to a date, and **J2**, it points at that day's actual plan rather than floating free.

Why `dayPlanId` is optional and not required. Some days will have no plan, and a person may still want to write something. Requiring the link would force them to invent a plan first. Optional also means a `DayPlan` can be discarded later without stranding entries.

What this table deliberately leaves out. No `promptId`, no coach transcript, no gratitude field, no images, no search index. `Mood` is a small fixed set tapped once, not free text, because a five-tap row gets used and a typing box does not. Each of those omissions maps to a PRD requirement that is deferred, and every one is a column or a screen added later — none of them change this table's shape.

Why the mood scale is worded the way it is. Two decisions inside that enum, both of which are easy to get wrong:

- **Five points, not four.** An even number of options has no true middle, so half the people answering are pushed toward a positive or negative answer they do not feel. `FLAT` in the middle is the honest answer and needs somewhere to sit.
- **No failure words.** `STRUGGLING`, `BAD`, and `TERRIBLE` would each break the PRD's tone rules, which ban red, "you failed", and any count of missed days. `ROUGH` says the day was hard without grading the person. This is also why the mood scale is a single-colour ramp in the

design system rather than green-to-red: **mood is not performance**. A rough day is not a failed day, and the interface must never imply that it was.

On storage. Per the revised Decision 6 in the implementation plan, `body` is stored as ordinary readable text and is **not** encrypted client-side in the prototype. Testers are told this plainly. Client-side encryption ships before any real user, as opposed to a tester, writes an entry.

2.4 How times are stored, and why

This is the single most common source of bugs in calendar software, so it is worth being precise.

Two different kinds of "time" exist, and they are stored differently.

1. Moments. A meeting from 14:00 to 15:00 on Tuesday. This is a specific instant on the timeline of the universe. It is stored as a real timestamp in UTC, behind the scenes. That means: when the user says "14:00", the app converts that to a UTC moment using the user's time zone from the `user` table. When they look at it again, it converts back to "14:00".

2. Patterns. "I work 9 to 5." "I am sharpest between 9 and 11." These are not moments. They are rules that repeat. Storing them as UTC timestamps would be a category error, and would break twice a year when the clocks change. So they are stored as plain numbers in the user's own local time — minutes from midnight, and hour of the day — and interpreted in their zone whenever they are needed.

Why it matters: if the user flies from London to New York, their meetings stay at the same real moments, and their working hours and energy curve follow them to their new local time. If you stored everything as UTC-shaped text, that would silently break.

One consequence you should know about: daylight saving can make a local day 23 or 25 hours long. The scheduler must clamp rather than crash when a task would run past the end of the day because of it. This is a real test case on day 9.

2.5 The four structural guarantees

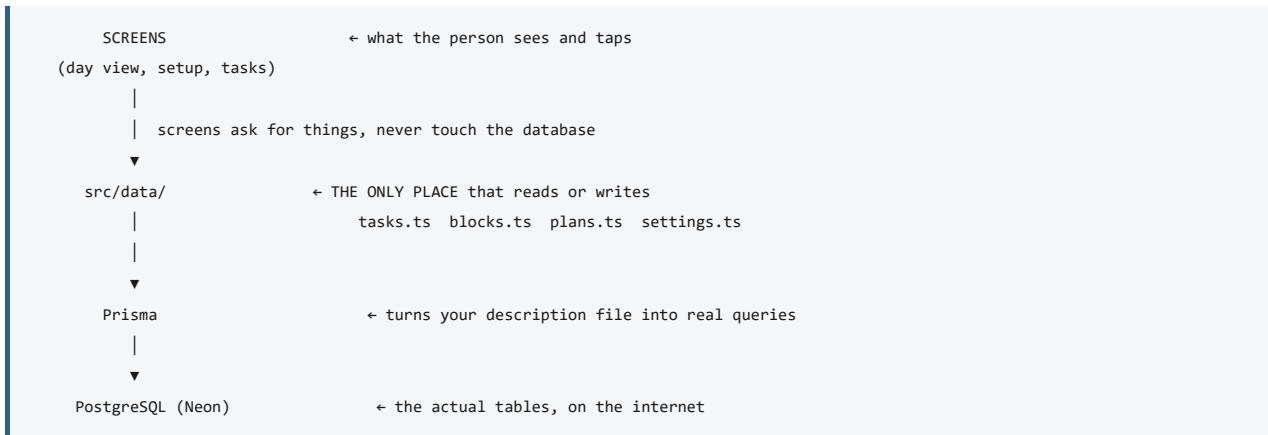
These are the properties that make the prototype a foundation. Each one is enforced by the database or the code shape, not by good intentions.

Guarantee	How it is enforced
A fixed meeting can never be a task	<code>Block</code> has no task column. There is nowhere to put the link.
A hand-typed meeting and a calendar meeting behave identically	<code>source</code> is history only. The scheduler never reads it.
Explanations can never go stale	Reasons are stored as facts; sentences are assembled at display time.
Your manual arrangements survive a re-plan	Pinned plan items are never moved.

3. How the pieces fit together

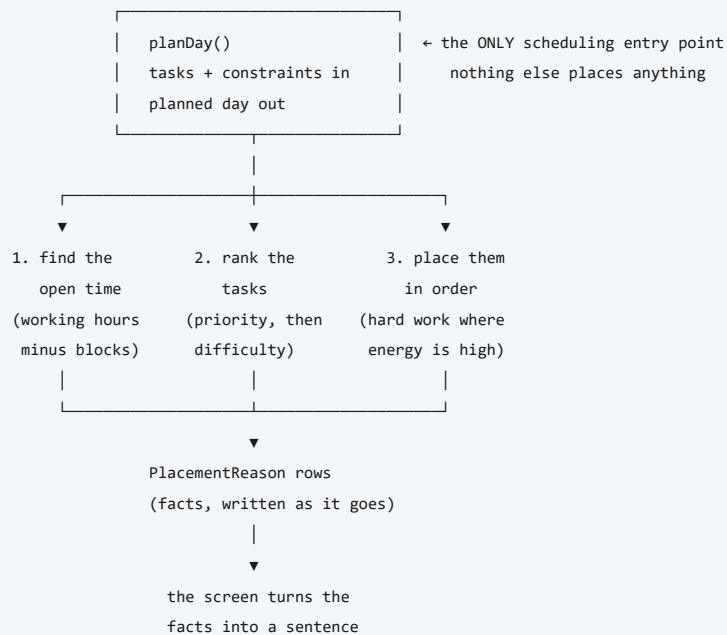
Two paths matter: **reading and writing**, and **making a plan**.

3.1 Reading and writing



The middle box is the important one. **No screen ever talks to the database directly.** Every read and write goes through `src/data/`. That is what makes offline support an addition later: you add a second implementation of the same small set of functions, point the app at it, and the screens do not change at all.

3.2 Making a plan



Three properties to hold on to:

- `planDay()` **is one function**. Every screen that needs a plan calls it. It takes tasks and constraints in, and returns a whole planned day. It has no idea a screen exists.
- **It is deterministic**. The same inputs always produce the same day. Not a better day — the *same* day. This matters more than clever scheduling, because if a user moves something once and it snaps back tomorrow, they stop trusting the entire product.
- **It never asks where a block came from**. It receives blocks and subtracts their time. A synced meeting and a typed lunch are handled by the same line of code.

4. Folder and file structure

```
focusflow/
|
├─ prisma/
|   └─ schema.prisma      ← your table descriptions. The database is
                           generated from this. You rarely edit it
                           after day one.
|
|
├─ src/
|   |
|   ├─ app/               ← SCREENS. What the person sees.
|   |   └─ page.tsx       ← the day view: today's timeline
|   |   └─ setup/page.tsx ← working hours + draw your energy curve
|   |   └─ tasks/page.tsx ← add and edit tasks
|   |   └─ blocks/page.tsx ← add meetings and fixed time
|   |   └─ plan/page.tsx  ← the proposed day, with reasons
|   |   └─ layout.tsx     ← shared frame around every screen
|   |
|   ├─ data/              ← THE ONLY PLACE THAT TOUCHES THE DATABASE
|   |   └─ client.ts      ← the one connection
|   |   └─ tasks.ts       ← getTasks, createTask, updateTask
|   |   └─ blocks.ts      ← getBlocks, createBlock
|   |   └─ plans.ts       ← savePlan, getPlan, acceptPlan
|   |   └─ settings.ts    ← working hours and energy curve
|   |   └─ index.ts       ← the short list other code is allowed to import
|   |
|   ├─ scheduling/        ← THE ONLY PLACE THAT ARRANGES TIME
|   |   └─ plan-day.ts     ← planDay(): tasks + constraints in, day out
|   |   └─ energy.ts       ← "how alert is this person at hour 14?"
|   |   └─ constraints.ts  ← open-time finding and ranking rules
|   |   └─ reasoning.ts    ← turns reason fields into readable sentences
|   |
|   └─ lib/
|       └─ time.ts         ← every timezone conversion, in one file
|       └─ types.ts        ← shared shapes, including the reason fields
|
|   └─ components/        ← small reusable pieces: TimeBlock, EnergyPicker, Button
|
├─ tests/
|   └─ plan-day.test.ts    ← the scheduler must be predictable
|   └─ time.test.ts       ← timezone and daylight-saving cases
|
└─ package.json
```

4.1 The three rules about this structure

Screens never import Prisma. If a screen needs data, it calls `src/data/`. This is what Section 5's offline row depends on.

Only `plan-day.ts` places things in time. If you find yourself sorting tasks inside a screen, that code has escaped and needs to move.

Only `time.ts` converts time zones. One file that knows about time zones means one place to look when a time is wrong.

4.2 Two folders that look empty and are not

`prisma/schema.prisma` contains `Goal` and `Project` models with no screens and no data behind them.

`src/app/` contains no goal or project screens.

That is intentional. **Both halves exist on day one** — the columns and the folder — so that switching the feature on is a matter of adding screens to a place that already exists, rather than reshaping data that people have already typed in.

5. Where each dropped feature plugs in later

For each feature you removed from the prototype: where it lands, and what it changes.

The column that matters is the last one. **Eight of the nine require no change to the existing design.**

Dropped feature	Where it plugs in	What has to change
Goals and projects	<code>Goal</code> and <code>Project</code> tables, already present and empty. New screens under <code>src/app/goals/</code> and <code>src/app/projects/</code> . Add a goal picker to the task form.	Nothing existing. New screens, new tables' first data. Rule A already left the columns in place.
Quick capture and inbox	<code>Task.state</code> already includes <code>INBOX</code> . A capture screen creates tasks with <code>state = INBOX</code> . An inbox screen is a list filtered on that one value. The scheduler skips anything not <code>READY</code> .	Nothing existing. One filter condition in the scheduler, written now.
Focus mode	New screen. The day's plan already knows which block is next. For a timer, a browser app is enough. For blocking other apps , native is required — this is the one feature that genuinely needs a different platform.	Nothing existing for the timer. The native app for true app-blocking.
Evening review and journal	<code>JournalEntry</code> and a minimum journal ship on Day 9 — see Section 2.2. The remaining work is prompts (J3), coach follow-ups (J7), search (J5), week/month views (J6) and export (J8). <code>DayPlan.state</code> already records <code>ACCEPTED</code> , so "did you accept the plan?" is already answerable.	Nothing existing. The minimum table is built now, so the additions are columns and screens on top of it.
Routines	<code>Task.recurrenceRule</code> already exists in standard <code>RRULE</code> text. When a day is planned, one function expands a recurring task into that day's occurrence.	Nothing existing in the schema. One new function, <code>expandRecurrence</code> .
Onboarding	New screens that write to <code>User</code> , <code>WorkingHour</code> , and <code>EnergyEntry</code> — all three tables already exist.	Nothing existing. Pure screen work, which is the cheapest kind.
Calendar sync	<code>Block.source</code> and <code>Block.externalId</code> already exist. A new service writes synced events as ordinary <code>Block</code> rows with <code>source = CALENDAR</code> .	Nothing in the scheduler. This is the direct payoff of Rule B — the scheduler already treats synced and typed time identically, so sync touches one direction only.
Progress and insights	Read-only screens over <code>Task</code> , <code>PlanItem</code> , and <code>PlacementReason</code> . <code>PlacementReason</code> is the gift here: because reasons were stored as facts, insights can query "every hard task placed during low-energy hours" directly.	Nothing existing in capture. Insights are queries, not new writes. Needs the Section 7 score formula.
Offline support	A second implementation of the <code>src/data/</code> functions, backed by local storage, syncing later. Screens are untouched because they only ever called <code>src/data/</code> .	One folder. This is the entire reason for that folder.

5.1 The honest ordering

Adding these is not all equal cost. Roughly, in order of easiest to hardest:

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.

That is the same order as the implementation plan's Section 1B, and the costs do not change.

6. Day-by-day build order

Roughly one task per day. Some days run long. That is normal and not a sign you are doing it wrong.

Each day ends with a test you can do yourself in a browser, in a few minutes. **If you cannot perform the day's test, the day is not finished** — even if the screen looks right.

Day 1 — The project runs, and it can store things

Build

- Create the Next.js project, install Prisma, connect to the hosted database
- Write `schema.prisma` from Section 2.3, including the unused `Goal` and `Project` models
- Create the database
- Write `src/data/client.ts` and `src/data/settings.ts`
- Build the setup screen: working hours, and an energy curve the person draws by tapping hours

Test at the end of the day

- 1.
- 2.
- 3.
- 4.

Why this test matters: step 4 is the first proof that the data layer works. Everything after this depends on it.

Day 2 — Tasks and fixed blocks can be typed in

Build

- `src/data/tasks.ts` and `src/data/blocks.ts`
- Tasks screen: add, edit, mark done
- Blocks screen: add a meeting with a start and end
- A time helper so entering "14:00" is a time picker, not typing

Test at the end of the day

- 1.
- 2.
- 3.
- 4.

Why this test matters: step 3 catches time-zone bugs on day 2 instead of on day 9.

Day 3 — The scheduler places tasks

Build

- `src/scheduling/constraints.ts` — find open time from working hours minus blocks
- `src/scheduling/energy.ts` — read the energy level for an hour
- `src/scheduling/plan-day.ts` — `planDay()`
- The version to write: list open gaps, rank tasks by priority then difficulty, and place each one in the earliest suitable gap. Hard tasks prefer high-energy hours.

Test at the end of the day

- 1.
- 2.
- 3.
- 4.
- 5.

Why this test matters: step 5 is determinism. If re-planning gives a different answer for identical input, every later feature inherits that unreliability.

Day 4 — It explains itself, and admits what did not fit

Build

- `src/scheduling/reasoning.ts` — assemble the sentence from stored fields
- `src/data/plans.ts` — save a `DayPlan` and its `PlacementReason` rows

- Plan screen: the proposed day, each block with its reason on tap
- An "unscheduled today" section

Test at the end of the day

- 1.
- 2.
- 3.
- 4.
- 5.

Why this test matters: this is Rule C being tested for the first time. If a sentence ever disagrees with its fields, a bug has crept into the reasoning code — catch it now.

Day 5 — The day view

Build

- `page.tsx` — today's timeline, top to bottom, blocks at their real times
- Distinguish visually: fixed blocks, scheduled work, free time
- `TimeBlock` component, then `Button` and `EnergyPicker` in `components/`

Test at the end of the day

- 1.
- 2.
- 3.
- 4.
- 5.

Why this test matters: this is the screen the product lives or dies on. If the day is unreadable, the scheduler's cleverness is wasted.

Day 6 — Accepting a day

Build

- Accept and discard actions, writing `DayPlan.state`
- Mark a plan item done, setting `Task.state = DONE` and `completedAt`
- A simple "today" summary

Test at the end of the day

- 1.
- 2.
- 3.
- 4.

Why this test matters: completes the loop. `DayPlan.state` is also what the future evening review will read, so this state machine is load-bearing.

Day 7 — Re-planning without losing your changes

Build

- `DayPlan.regenerateCount` and keeping the previous plan
- Re-plan flow: preserve every pinned item, place the rest again, show what changed

Test at the end of the day

- 1.
- 2.
- 3.
- 4.
- 5.

Why this test matters: step 3 is a promise about trust. Silently undoing a user's deliberate arrangement is the fastest way to lose them.

Day 8 — Moving a block by hand

Build

- Drag or nudge a block on the day view, setting `isPinned`
- A "what did I break?" check: if a move overlaps a meeting, say so plainly rather than allowing it
- Empty states: no tasks, no blocks, nothing scheduled, nothing unscheduled (*moved here from Day 9, so the Day 9 journal has room*)

Test at the end of the day

- 1.
- 2.
- 3.
- 4.
- 5.

Why this test matters: step 1 protects the core invariant. Overlapping time is the one thing this product must never produce. Step 5 protects a different thing: a first-run user who sees four empty panels concludes the app is broken.

Day 9 — The evening review, and the awkward cases

Build

- **JournalEntry and the evening review.** Read `DayPlan`, `PlanItem`, and `Task` for today and show what actually happened: what was planned, what got done, what did not fit. One free-form text box and a five-tap mood row. Save.
- A read-only list of past entries, newest first. No search, no prompts, no editing UI beyond fixing today's entry.
- Run the app as a real production build on your computer: `npm run build`, then `npm start`
- Open a free Cloudflare Tunnel link so testers can reach it from anywhere
- The daylight-saving guard from Section 2.4: clamp, never crash
- `tests/time.test.ts` and `tests/plan-day.test.ts`

Test at the end of the day

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

Why this test matters: day 9 is where prototypes usually fail — real phones, real time zones. Testing these with testers tomorrow is avoidable embarrassment.

Why the journal is on Day 9 specifically, and not later. Day 10 is the tester session. Anything not built by the end of Day 9 does not get tested by a real person, and PRD risk 8 says plainly that "*journaling is abandoned in week two.*" The question this build answers is not whether journaling is well-built — it is **whether anyone writes in it at all**. A simple version that people actually use beats a complete version that ships after the testers have gone.

What this deliberately leaves out, and why that is correct. No guided prompts (J3) and no coach follow-ups (J7) — both need the AI coach, which is not in this prototype. No search (J5), week or month views (J6), or export (J8). Each is a screen or a column added on top of `JournalEntry`, and none of them change the table's shape. Adding them later is work, not repair.

Note on running it for real. While we test, the app is only alive while your computer is on. That is fine for a handful of testers. The stack audit, Part 1 item 7, covers when that stops being true and what it costs to fix.

Day 10 — The real test

Build

- Nothing new. Resist it.
- Write the one-line instruction you will read to each tester

Test at the end of the day

- 1.
- 2.
- 3.
- 4.

The number that decides this: under 30% of blocks moved by hand.

Read that as: *in most cases, the app got the day right well enough that the person left it alone*. Below 30% and the thesis holds — build the rest. Above 30% and the thesis is not yet proven; the plan stays, and the feedback tells you which part of the reasoning was wrong.

Either outcome is useful. Only not testing is useless.

7. The two Phase 0 decisions

Two open questions block the work. Both need an answer before day 3. Neither takes long.

Decision 1 — What does "consistency" actually measure?

The problem: later, the app will show each person a score, and possibly rank people against each other. A score measuring the wrong thing is worse than no score, because it teaches the wrong lesson.

Option	What it measures	The problem
A. Tasks completed	Completed ÷ scheduled	Measures busyness, not progress. Someone scheduling trivia scores beautifully.
B. Goals advanced	Completed work that moved an active goal forward	Honest. But on day one you have no goals, so there is no history.
C. Ritual adherence	Did you plan in the morning, review at night	Predicts whether someone sticks around, but rewards showing up over accomplishing.
D. A blend	All three	Balanced, but the weights are arbitrary and you cannot explain them in one sentence.

My recommendation: Option B. With this sentence, which is short enough to show a user:

Your consistency is the share of your scheduled work you completed that moved one of your active goals forward.

In plain words: the app does not reward you for being busy. It rewards you for finishing the things you said mattered. That is the entire argument of this product, so the score should measure exactly that argument, not a diluted version of it.

What this means for the prototype: nothing to build now. But this is exactly why `Task.goalId` sits empty from day one — **Rule A is what makes this decision cheap**. The moment you turn on goals, the data needed for this score already exists. Had the prototype had no goal link, switching on goals later would mean asking every user to re-file their history.

What I need from you: confirm or overrule, and write the sentence down. It has to be reproducible from stored data, not remembered.

Decision 2 — Where does the energy signal come from?

The problem: matching hard work to alert hours is the thing that makes this app different from a to-do list. But where does the app learn how alert you are?

Option	Where the signal comes from	The problem
A. You say so	The person draws their own curve in setup	Available instantly. Never updates — but real energy shifts week to week.
B. The app watches	Learns from which tasks you finish, and when	Genuinely accurate. Needs three or four weeks of use before it knows anything.
C. Both	Starts from what you said, adjusts from what you did	Best. Most work.

My recommendation: Option A for the prototype, structured so Option C arrives later.

That is a deliberate change from the long-term plan, which recommends C. Here is the reasoning.

Option B is fatal for a prototype. A new user has no history, and the first generated day is the exact moment belief forms. If that first day is bad because there is no data yet, they never get far enough for the data to help. Option A costs one screen and works on the very first use.

Option C's real problem is not difficulty, it is **trust**. If the app silently decides your energy pattern is different from what you told it, and you cannot see why, you stop trusting the schedule. So whenever observed data is added, it must be shown to you, and reversible.

The schema already supports this. `EnergyEntry.source` is `DECLARED` for every row on day one. When observed learning arrives, it writes new rows with `source = OBSERVED`. No restructure, no migration of your existing settings.

The recommendation for later, so it is not decided under pressure: observed data wins once there is enough of it to be meaningful — say, thirty completed tasks — and when it does, the app tells you plainly: *"We moved your deep work to mornings because you have actually been finishing it there."* You can always revert to your declared curve.

What I need from you: confirm you want declared-only for now, and confirm you are happy for the curve to be treated as an assumption rather than a fact in the interface.

8. What could still go wrong

Honest list, so nothing here is a surprise.

Risk	How bad	What we do about it
The browser app cannot block other apps	Annoying, not fatal	Focus mode ships as a timer in the browser. True app-blocking needs native later. The rest of the app moves across.
Offline support is harder from a web app	Real	This is the main cost of the stack choice, and it is why <code>src/data/</code> exists. Adding it means one new folder, not a rewrite.
The scheduler is too clever	Fatal to trust	Guarded by determinism (day 3) and by preferring predictable over optimal.
Ten days runs long	Normal	You will lose a day to something small. That is why days 1, 2, and 9 have explicit fallback versions.
Real time zones break something	Real, and sneaky	Tested explicitly on day 9, not discovered in a beta.
You outgrow the plan and lose momentum	Common	Every day ends in something you can click. No day ends in "infrastructure".
Testers are not your target users	Real	Five friendly testers is a signal, not a verdict. Read <i>why</i> they moved things, not just how often.

9. Sign-off

Five items were put to you; four are confirmed and one is still open. They are one afternoon's work and they prevent a rebuild.

Item	Recommendation	Confirmed
Stack	Browser app: Next.js, TypeScript, Prisma, PostgreSQL, hosted on your computer	Yes — local hosting accepted over Vercel, 4 Oct 2026
Decision 1	Score measures goals advanced, not tasks completed	Yes — 4 Oct 2026
Decision 2	Declared energy curve now, observed later, disclosed when added	Yes — 4 Oct 2026
Offline deferred	Accept the tradeoff; <code>src/data/</code> keeps the door open	<i>Open</i> — implied by choosing local hosting with a remote database, but never confirmed in so many words
Test threshold	Under 30% of blocks moved by hand	<i>Open</i> — never discussed

Journal decisions, added after review

Item	Decision	Confirmed
Journal scope	Minimum journal on Day 9; full journal deferred	Yes — 4 Oct 2026
Journal storage	Plain readable text in Neon, no client-side encryption in the prototype	Yes — 4 Oct 2026
Encryption trigger	Ships before any real user, as opposed to a tester, writes an entry	Yes — 4 Oct 2026
Mood scale	Five points, single-colour ramp, no failure words	Yes — <code>GREAT GOOD OK FLAT ROUGH</code> , 4 Oct 2026

Known watch item, no change made. `Outcome` is `PLACED / REJECTED`. `REJECTED` is the same class of word as `STRUGGLING` was, and the PRD's tone rules ban "you failed" language. It was reviewed and deliberately left alone, because the subject is a *proposed plan* being turned

down rather than the person — the same distinction the mood scale relies on. Worth re-reading once real screens exist, since a button reading "Reject" is harsher in the hand than it is in a schema.

Immediate next action: answer the two open items above, then build Day 1. Day 1 is: the project runs, the database exists, and the setup screen saves your settings after a browser restart.